

Proyecto SO

Proyecto final de Sistemas Operativos

Implementación de un servidor de chat multiciente en C++ sobre Raspberry Pi con autenticación, roles, bitácora de auditoría, contraseñas SHA-256 y protección mediante firewall

Versión actualizada

1. Introducción

El presente proyecto consiste en el desarrollo de un sistema de comunicación local de tipo cliente-servidor, implementado en C++ sobre un sistema operativo GNU/Linux, ejecutado en una Raspberry Pi. El sistema permite la conexión simultánea de múltiples usuarios a un servidor de chat dentro de una red local.

La versión actualizada incorpora autenticación de usuarios, autorización por roles, mensajería pública y privada, administración de usuarios, bitácora profesional de eventos, protección básica de credenciales mediante SHA-256, cliente con IP configurable y protección del servicio mediante firewall.

El proyecto se desarrolló con el propósito de aplicar conceptos fundamentales de Sistemas Operativos, como sockets TCP, concurrencia con hilos, sincronización con mutex, manejo de archivos, administración remota por SSH, seguridad básica y despliegue de servicios en un entorno Unix real.

2. Resumen del proyecto

Este proyecto atiende la necesidad de contar con un sistema de mensajería local que funcione dentro de una red interna, sin depender de servicios externos ni de conexión a internet. Está dirigido a estudiantes, administradores de sistemas o usuarios que deseen aprender sobre programación de redes, concurrencia, administración de usuarios y seguridad básica en entornos Unix.

La finalidad del proyecto es integrar, en una sola solución funcional, conceptos de Sistemas Operativos y redes. Se eligió C++ por su cercanía al sistema, su compatibilidad con llamadas de bajo nivel en Linux y su soporte de hilos mediante `std::thread`. Se utilizaron sockets POSIX para implementar comunicación TCP/IP, GNU Readline para mejorar la experiencia del cliente en terminal, OpenSSL para generar hashes SHA-256 y UFW para restringir los puertos expuestos.

3. Tiempo estimado de desarrollo

Tiempo total estimado de desarrollo del proyecto actualizado: 32 horas.

Actividad	Tiempo aproximado
Planeación general del proyecto	2 horas
Investigación sobre sockets y arquitectura cliente-servidor	3 horas
Implementación del servidor básico	3 horas
Implementación del cliente básico	2 horas
Soporte multicliente con hilos	3 horas
Implementación de autenticación y archivo de usuarios	2 horas
Comandos del sistema (/usuarios, /msg, /registrar, /expulsar, /salir)	4 horas
Comando /ayuda y mejora de comandos disponibles	1 hora
Cliente con IP y puerto configurables por terminal	1 hora
Sistema de roles admin/usuario	2 horas
Integración de autocompletado con readline	2 horas
Bitácora profesional de eventos	2 horas
Hash de contraseñas con SHA-256 y OpenSSL	2 horas
Configuración de firewall y pruebas de red	2 horas
Pruebas, corrección de errores y ajustes finales	3 horas

4. Tecnologías usadas

- GNU/Linux como sistema operativo base
- Raspberry Pi como servidor físico
- C++ como lenguaje principal de programación
- Sockets POSIX para comunicación TCP/IP

- `std::thread` para concurrencia multicliente
- `std::mutex` para sincronización de datos compartidos
- GNU Readline para historial y autocompletado en el cliente
- OpenSSL para generación de hash SHA-256
- UFW para protección del servicio mediante firewall
- SSH para administración remota del servidor
- `nmap` y `netcat` para pruebas de red y validación de puertos

5. Librerías utilizadas

5.1 Librerías estándar de C++

- `iostream`
- `thread`
- `vector`
- `string`
- `cstring`
- `algorithm`
- `mutex`
- `fstream`
- `ctime`
- `sstream`
- `unordered_map`
- `atomic`
- `iomanip`

5.2 Librerías del sistema Unix/Linux

- `unistd.h`
- `arpa/inet.h`
- `sys/socket.h`
- `netinet/in.h`

5.3 Librerías adicionales

- `readline/readline.h`
- `readline/history.h`
- `openssl/sha.h`

5.4 Motivo de uso de las librerías

Las librerías estándar de C++ permitieron manejar estructuras dinámicas, archivos, hilos, cadenas de texto, formateo de salida y sincronización entre hilos. Las cabeceras de sockets de Linux permitieron implementar la comunicación real entre cliente y servidor mediante TCP/IP. `readline` se utilizó para contar con una interfaz de terminal más usable, con historial y autocompletado mediante `Tab`. `mutex` fue necesario para proteger el acceso concurrente a estructuras compartidas. `fstream` se usó para persistir usuarios y guardar la bitácora del sistema. OpenSSL se incorporó para transformar contraseñas en hashes SHA-256 antes de compararlas o almacenarlas.

6. Lenguaje de programación utilizado

El proyecto fue desarrollado en C++.

Se eligió C++ porque permite trabajar con abstracciones de alto nivel y, al mismo tiempo, conserva acceso directo a herramientas del sistema operativo como sockets, archivos, memoria, hilos y sincronización. Es un

lenguaje apropiado para comprender cómo un programa interactúa con GNU/Linux y cómo puede construir servicios concurrentes sobre Unix.

7. Herramientas extra utilizadas

- g++ para compilación
- SSH para acceso remoto a la Raspberry Pi
- UFW para administración del firewall
- nmap para escaneo y verificación de puertos
- netcat (nc) para pruebas directas de conexión TCP
- nano o editor de texto en terminal para modificar archivos
- sha256sum para validar hashes desde la terminal
- systemd como mejora futura para ejecutar el servidor como servicio
- Raspberry Pi OS / Debian-based GNU/Linux como entorno de ejecución

8. Objetivo general

Desarrollar un sistema de chat local multicliente en C++ sobre una Raspberry Pi con GNU/Linux, implementando comunicación TCP, autenticación de usuarios, autorización por roles, administración de sesiones concurrentes, registro de eventos, protección de credenciales mediante SHA-256 y protección del servicio mediante firewall.

9. Objetivos específicos

- Implementar un servidor TCP concurrente que acepte múltiples clientes.
- Permitir autenticación mediante usuario y contraseña almacenados en archivo.
- Guardar contraseñas como hash SHA-256 en lugar de texto plano.
- Evitar que un mismo usuario se conecte simultáneamente más de una vez.
- Incorporar roles de usuario para separar permisos administrativos y permisos normales.
- Incorporar comandos administrativos y de interacción para el chat.
- Implementar mensajería privada entre usuarios conectados.
- Registrar eventos del sistema en un archivo de auditoría chat.log.
- Restringir el acceso a puertos mediante firewall.
- Mejorar la experiencia de uso del cliente con autocompletado por terminal.
- Permitir que el cliente reciba IP y puerto del servidor por argumentos de terminal.

10. Descripción funcional del sistema

10.1 Servidor

El servidor corre en la Raspberry Pi y permanece escuchando conexiones en el puerto 5000. Cada vez que un cliente se conecta, el servidor crea un hilo independiente para atenderlo. El servidor administra la autenticación, controla la lista de clientes conectados, procesa comandos, valida permisos por rol, registra eventos y reenvía mensajes al resto de los usuarios.

10.2 Cliente

El cliente se ejecuta desde otra máquina dentro de la misma red local. El usuario indica la IP y el puerto del servidor al ejecutar el programa. Después ingresa su nombre y contraseña. Si la autenticación es correcta, accede al chat. Desde ahí puede enviar mensajes públicos, mensajes privados, consultar usuarios conectados y, si tiene rol de administrador, registrar o expulsar usuarios.

11. Funcionalidades implementadas

- Conexión cliente-servidor mediante TCP
- Soporte multicliente
- Cliente con IP y puerto configurables por terminal
- Autenticación por usuario y contraseña
- Contraseñas almacenadas como hash SHA-256
- Prevención de usuarios duplicados conectados
- Sistema de roles: admin y usuario
- Mensajes públicos
- Mensajes privados mediante comando /msg
- Consulta de usuarios conectados mediante /usuarios
- Menú de ayuda mediante /ayuda
- Registro de nuevos usuarios mediante /registrar usuario contraseña rol
- Expulsión de usuarios mediante /expulsar
- Salida del chat mediante /salir
- Actualización dinámica de usuarios conectados
- Autocompletado de comandos y nombres de usuarios en terminal
- Bitácora profesional en archivo chat.log
- Restricción de puertos con firewall

12. Comandos disponibles en el sistema

Comando	Función	Permiso requerido
/ayuda	Muestra la lista de comandos disponibles.	admin y usuario
/usuarios	Muestra la lista de usuarios conectados, su rol e IP.	admin y usuario
/msg usuario mensaje	Envía un mensaje privado a un usuario conectado.	admin y usuario
/registrar usuario contraseña rol	Registra un nuevo usuario con rol admin o usuario.	solo admin
/expulsar usuario	Expulsa a un usuario conectado.	solo admin
/salir	Cierra la sesión y desconecta al cliente.	admin y usuario

13. Usuarios, roles y permisos

El sistema de chat maneja dos roles básicos de acceso: usuario y admin. El rol se define en el archivo usuarios.txt. Ya no depende de que un usuario tenga un nombre fijo en el código, sino de su rol.

Rol	Puede hacer	No puede hacer
usuario	Autenticarse, enviar mensajes públicos, enviar mensajes privados, consultar usuarios conectados, usar /ayuda y salir del sistema.	Registrar usuarios o expulsar usuarios conectados.
admin	Todas las funciones del usuario normal, además de registrar nuevos usuarios y expulsar usuarios conectados.	No puede expulsarse a sí mismo.

14. Archivo usuarios.txt actualizado

El archivo usuarios.txt utiliza el formato:

```
usuario:hash_sha256:rol
```

Ejemplo de usuarios con contraseña hasheada:

```
faisan:ca261d732cc4d2fc9d3d287e9bb961c217ee1b14648693ab024c1d71096eda4b:admin
```

```
juan:03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4:usuario
```

```
maria:03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4:usuario
```

En este ejemplo, la contraseña de faisán es neri. La contraseña de Juan y María es 1234. El usuario escribe su contraseña normal en el cliente; el servidor calcula el hash SHA-256 y lo compara contra el valor almacenado.

15. Seguridad implementada

- Autenticación obligatoria antes de ingresar al chat.
- Contraseñas almacenadas como hash SHA-256 en usuarios.txt.
- Restricción de usuarios duplicados.
- Control de privilegios mediante roles.
- Comandos administrativos limitados al rol admin.
- Registro de actividad y errores en chat.log.
- Firewall configurado para permitir solo puertos necesarios.
- Control de permisos del sistema Linux para evitar que usuarios no autorizados modifiquen el servidor.

15.1 Puertos permitidos

- 22/tcp para SSH
- 5000/tcp para el servidor de chat

15.2 Puertos bloqueados

Todos los demás puertos entrantes no autorizados se mantienen bloqueados mediante firewall.

15.3 Consideración de seguridad

SHA-256 evita almacenar las contraseñas en texto plano, pero el tráfico del chat aún no se encuentra cifrado. Como mejora futura, se podría integrar TLS para proteger las credenciales durante el tránsito por la red.

16. Bitácora profesional del sistema

El servidor registra eventos en el archivo chat.log con un formato de auditoría. Esto permite revisar conexiones, autenticaciones, mensajes, comandos, errores y acciones administrativas.

Formato general del log:

```
[YYYY-MM-DD HH:MM:SS] [TIPO_EVENTO] detalle=valor
```

Eventos registrados:

- SERVER_START
- CONNECTION
- LOGIN_OK
- LOGIN_FAIL
- LOGIN_DUPLICADO
- LOGOUT
- MSG_PUBLICO
- MSG_PRIVADO
- MSG_PRIVADO_FAIL
- USER_CREATED

- KICK
- COMMAND
- COMMAND_ERROR
- COMMAND_UNKNOWN
- PERMISSION_DENIED
- ERROR

Ejemplo:

```
[2026-05-27 13:55:20] [CONNECTION] ip=10.255.89.38 estado=nueva_conexion
[2026-05-27 13:55:25] [LOGIN_OK] usuario=faisan rol=admin ip=10.255.89.38
[2026-05-27 13:56:10] [USER_CREATED] admin=faisan nuevoUsuario=pedro rol=usuario
[2026-05-27 13:57:20] [MSG_PUBLICO] usuario=juan longitud=15
[2026-05-27 13:58:00] [KICK] admin=faisan expulsado=juan ip=10.255.89.40
```

17. Desarrollo del proyecto: pasos y tareas realizadas

Etapa 1. Planeación

Se definió el objetivo del sistema: construir una aplicación cliente-servidor sobre Linux que permitiera observar, de manera práctica, el comportamiento de procesos concurrentes, sockets y administración de recursos del sistema.

Etapa 2. Configuración del entorno

Se identificó la IP de la Raspberry Pi dentro de la red local y se habilitó la administración por SSH. También se verificó el compilador g++, OpenSSL, readline y las herramientas necesarias.

Etapa 3. Desarrollo del servidor base

Se programó un socket TCP que escuchara conexiones en un puerto fijo. Posteriormente, se implementaron `bind()`, `listen()` y `accept()` para crear el ciclo de recepción de clientes.

Etapa 4. Desarrollo del cliente base

Se implementó un cliente que se conecta al servidor mediante IP y puerto, envía mensajes y recibe respuestas.

Etapa 5. Cliente configurable

Se eliminó la IP fija del cliente y se permitió recibir la IP y el puerto como argumentos desde terminal.

Etapa 6. Soporte multicliente

Se integró `std::thread` en el servidor para atender a cada cliente en un hilo independiente.

Etapa 7. Sincronización y control compartido

Debido a que varios hilos acceden a la lista de clientes conectados, se incorporó `std::mutex` para evitar condiciones de carrera y corrupción de datos.

Etapa 8. Autenticación y roles

Se creó un archivo `usuarios.txt` con el formato `usuario:hash_sha256:rol`. El servidor valida credenciales y permisos según el rol.

Etapa 9. Administración de comandos

Se agregaron comandos para visualizar usuarios, enviar mensajes privados, registrar nuevos usuarios, expulsar usuarios conectados, salir del chat y consultar ayuda.

Etapa 10. Interfaz de terminal mejorada

Se integró la librería readline para permitir historial y autocompletado de comandos y usuarios.

Etapa 11. Bitácora y auditoría

Se actualizó chat.log para registrar eventos estructurados con fecha, tipo de evento y detalles relevantes.

Etapa 12. Protección de credenciales

Se incorporó OpenSSL para calcular SHA-256 de las contraseñas y evitar almacenarlas en texto plano.

Etapa 13. Seguridad de red

Se configuró UFW para exponer únicamente el puerto de SSH y el puerto del chat.

Etapa 14. Pruebas

Se realizaron pruebas con múltiples clientes simultáneos, login correcto e incorrecto, usuarios duplicados, mensajes privados, acciones administrativas, logs y verificación del firewall mediante nmap.

18. Estructura lógica del proyecto

```
proyecto_so/  
├── servidor.cpp  
├── cliente.cpp  
├── usuarios.txt  
└── chat.log
```

Archivo	Descripción
servidor.cpp	Implementa la lógica del servidor, sockets, autenticación, roles, concurrencia, comandos, logs y administración de clientes.
cliente.cpp	Implementa el acceso del usuario al chat, conexión por IP y puerto, recepción de mensajes, historial y autocompletado.
usuarios.txt	Almacena usuarios, hashes SHA-256 de contraseñas y roles.
chat.log	Bitácora de eventos y actividad del sistema.

19. Comportamiento lógico del sistema

1. El servidor se inicializa y comienza a escuchar conexiones en el puerto 5000.
2. Un cliente se conecta al servidor usando la IP de la Raspberry Pi y el puerto 5000.
3. El servidor solicita usuario y contraseña.
4. El servidor calcula el SHA-256 de la contraseña ingresada.
5. El servidor compara el hash generado con el hash almacenado en usuarios.txt.
6. Si la autenticación es válida y el usuario no está conectado, el cliente entra al chat.
7. El servidor crea un hilo dedicado a ese cliente.
8. El usuario puede enviar mensajes o ejecutar comandos.
9. El servidor procesa cada mensaje según corresponda y valida permisos si el comando es administrativo.
10. Cada evento relevante se registra en chat.log.
11. Cuando el usuario sale o es expulsado, la conexión se cierra y se actualiza la lista de conectados.

20. Diagrama de flujo del sistema

```
Inicio
|
v
Iniciar servidor
|
v
Escuchar conexiones en puerto 5000
|
v
¿Se conecta un cliente?
|---- No ----> Seguir escuchando
|
Sí
|
v
Solicitar usuario y contraseña
|
v
Calcular SHA-256 de la contraseña ingresada
|
v
¿Hash coincide con usuarios.txt?
|---- No ----> Rechazar conexión y registrar LOGIN_FAIL
|
Sí
|
v
¿Usuario ya está conectado?
|---- Sí ----> Rechazar conexión y registrar LOGIN_DUPLICADO
|
No
|
v
Agregar cliente a lista de conectados
|
v
Crear hilo para atender al cliente
|
v
¿Recibe mensaje?
|---- No ----> Cerrar conexión y registrar LOGOUT
```

```

|
Sí
|
v
¿Es comando?
|---- No ----> Reenviar mensaje y registrar MSG_PUBLICO
|
Sí
|
v
Procesar comando y validar rol si aplica
|
v
Registrar evento en chat.log
|
v
Volver a esperar mensaje

```

21. Diagrama UML simplificado

```

+-----+
| Cliente |
+-----+
| - socketCliente |
| - ipServidor    |
| - puertoServidor |
| - usuariosConectados |
+-----+
| + conectar()    |
| + autenticar()  |
| + enviarMensaje() |
| + recibirMensajes() |
| + autocompletar() |
+-----+
|
v
+-----+
| Servidor |
+-----+
| - servidor_fd |
| - clientes[]  |
| - credenciales |
| - mutex       |

```

```

+-----+
| + iniciar()      |
| + aceptarClientes() |
| + manejarCliente() |
| + autenticarUsuario() |
| + hashSHA256()    |
| + enviarATodos()  |
| + enviarPrivado() |
| + registrarUsuario() |
| + expulsarUsuario() |
| + guardarEvento() |
+-----+
      |
      v
+-----+
| Usuario          |
+-----+
| - hashContraseña |
| - rol            |
+-----+

```

22. Justificación técnica

Este proyecto fue seleccionado porque integra directamente varios temas de Sistemas Operativos en una aplicación funcional. El modelo cliente-servidor permite estudiar el uso de sockets, mientras que la atención concurrente de múltiples usuarios evidencia el uso de hilos y sincronización. La autenticación, los roles y la bitácora complementan la administración y seguridad del servicio.

La implementación sobre una Raspberry Pi con GNU/Linux permitió trabajar en un entorno Unix real, cercano a contextos de administración de sistemas y despliegue de servicios. Además, la incorporación de SHA-256 mediante OpenSSL permite explicar una mejora concreta en el manejo de credenciales.

23. Pruebas realizadas

- Escaneo de red para localizar la IP de la Raspberry.
- Verificación del puerto 5000 con nmap.
- Prueba de conexión con netcat.
- Inicio de múltiples clientes concurrentes.
- Conexión del cliente desde laptop hacia Raspberry Pi usando IP configurable.
- Intentos de acceso con credenciales incorrectas.
- Validación de bloqueo de usuarios duplicados.
- Validación de contraseñas hasheadas con SHA-256.
- Envío de mensajes públicos.
- Envío de mensajes privados.
- Uso del comando /ayuda.
- Consulta de usuarios conectados con /usuarios.
- Registro de usuarios desde un administrador.
- Bloqueo de /registrar cuando lo intenta un usuario normal.

- Expulsión de usuarios conectados.
- Verificación de logs en chat.log.
- Verificación del firewall.
- Validación del autocompletado con Tab.

24. Resultados esperados

- Mantener comunicación estable entre múltiples clientes.
- Permitir acceso únicamente a usuarios autorizados.
- Impedir la duplicidad de sesiones del mismo usuario.
- Permitir administración básica por parte de usuarios con rol admin.
- Guardar contraseñas en formato hash SHA-256.
- Registrar correctamente la actividad del sistema.
- Limitar el acceso externo mediante reglas del firewall.
- Permitir que el cliente se conecte a distintas IP sin recompilar el código.

25. Limitaciones actuales

- El sistema se limita a una red local.
- La interfaz del cliente es de terminal.
- No existe cifrado de tráfico mediante TLS.
- Las contraseñas se almacenan hasheadas, pero aún no se usa salt individual por usuario.
- No se ha integrado aún como servicio permanente con systemd.
- La persistencia se realiza en archivos de texto, no en una base de datos.
- El sistema de roles aún es básico y solo maneja admin y usuario.

26. Posibles mejoras futuras

- Agregar salt individual a cada contraseña antes de aplicar SHA-256.
- Integrar TLS para cifrar el tráfico cliente-servidor.
- Integrar systemd para arranque automático del servidor.
- Crear cliente con interfaz gráfica.
- Agregar persistencia en SQLite.
- Agregar rotación de logs.
- Implementar roles más completos, como invitado, moderador y administrador.
- Agregar intentos máximos de login y bloqueo temporal.
- Agregar integración con GPIO de Raspberry Pi, por ejemplo LED de estado o pantalla OLED.

27. Conclusiones

El proyecto cumplió con el objetivo de implementar un sistema de chat multicliente sobre Unix/Linux utilizando C++. Se logró integrar comunicación TCP, concurrencia, autenticación, administración de usuarios, permisos por rol, protección básica de credenciales con SHA-256, bitácora de eventos y mecanismos elementales de seguridad dentro de una arquitectura cliente-servidor funcional.

El desarrollo permitió aplicar conocimientos propios de Sistemas Operativos en un contexto realista y práctico, especialmente en sockets, procesos ligeros, sincronización, archivos, permisos de Linux y servicios de red. Además de ser una herramienta funcional, el proyecto representa una base sólida para seguir explorando temas más avanzados como servicios persistentes, cifrado de tráfico, bases de datos, auditoría avanzada y despliegue profesional en sistemas Unix.

28. Anexo de compilación y ejecución

28.1 Instalar dependencias

```
sudo apt update  
sudo apt install libreadline-dev libssl-dev
```

28.2 Compilar servidor

```
g++ -std=c++17 -Wall -Wextra servidor.cpp -o servidor -pthread -lssl -lcrypto
```

28.3 Compilar cliente

```
g++ -std=c++17 -Wall -Wextra cliente.cpp -o cliente -pthread -lreadline
```

Si es necesario, también puede compilarse agregando ncurses:

```
g++ -std=c++17 -Wall -Wextra cliente.cpp -o cliente -pthread -lreadline -lncurses
```

28.4 Ejecutar servidor

```
./servidor
```

28.5 Ejecutar cliente

Si cliente y servidor están en la misma máquina:

```
./cliente
```

Si el servidor está en la Raspberry Pi y el cliente en una laptop:

```
./cliente IP_DE_LA_RASPBERRY 5000
```

Ejemplo:

```
./cliente 10.255.89.84 5000
```

28.6 Obtener IP de la Raspberry Pi

```
hostname -I
```

28.7 Consultar logs

```
cat chat.log  
tail -f chat.log
```

28.8 Generar hash SHA-256 desde Linux

```
echo -n "neri" | sha256sum  
echo -n "1234" | sha256sum
```